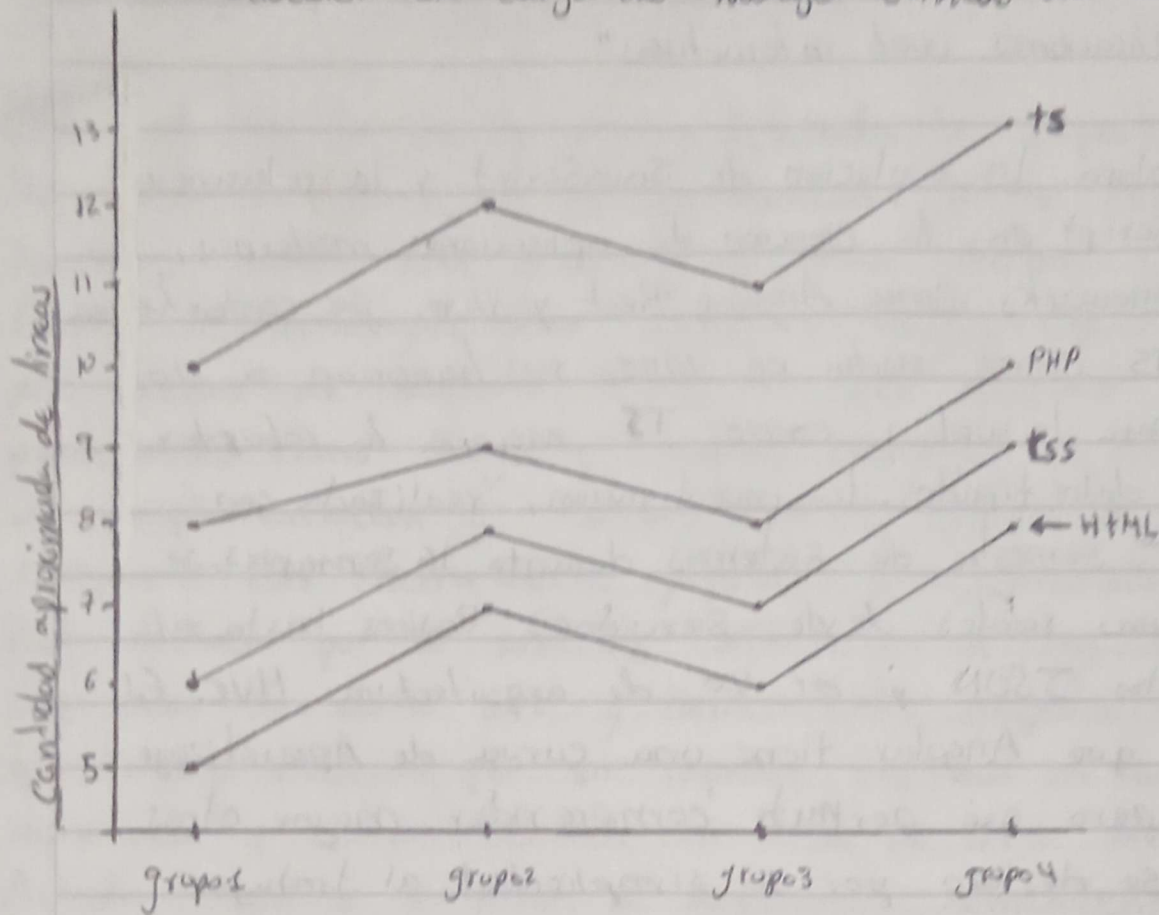


Cristofer David Iorano Contreras.

Resumen Artículo 1 "Frameworks basados en TypeScript para el desarrollo de aplicaciones web interactivas"

Este Artículo explora la evolución de JavaScript y la relevancia actual de TypeScript en la creación de aplicaciones modernas, destacando frameworks como Angular, React y Vue. Se contextualiza el origen de JS como moda en 1995, su transición a un lenguaje clave para la web y como TS mejora la robustez y el manejo de datos tipados. La investigación, realizada con estudiantes de 8^{vo} semestre de Sistemas durante 16 semanas, se enfocó en prácticas reales desde Servidores Básicos hasta el consumo de Datos JSON y el uso de arquitecturas MVC. El estudio mostró que Angular tiene una curva de Aprendizaje más amplia, pero que permite comprender mejor otros marcos; React se destacó por su simplicidad al trabajar directamente con HTML y TS, mientras que Vue resultó más intuitivo tras experiencias previas. El trabajo enfatiza que estas herramientas reducen tiempos y líneas de código, como lo evidencian las comparaciones de carga y dificultad de ejercicios, incluyendo tecnologías como .NET Core y Linq. Se concluye que los frameworks en TS no solo optimizan la productividad y calidad del código, sino que también facilitan la adaptación de los estudiantes a entornos laborales, aunque la investigación se centró principalmente en el frontend. Este análisis subraya el valor de metodologías prácticas y basadas en problemas para fortalecer competencias técnicas y preparar a los estudiantes para proyectos reales.

Reduccion de carga de trabajo (Lineas de codigo)

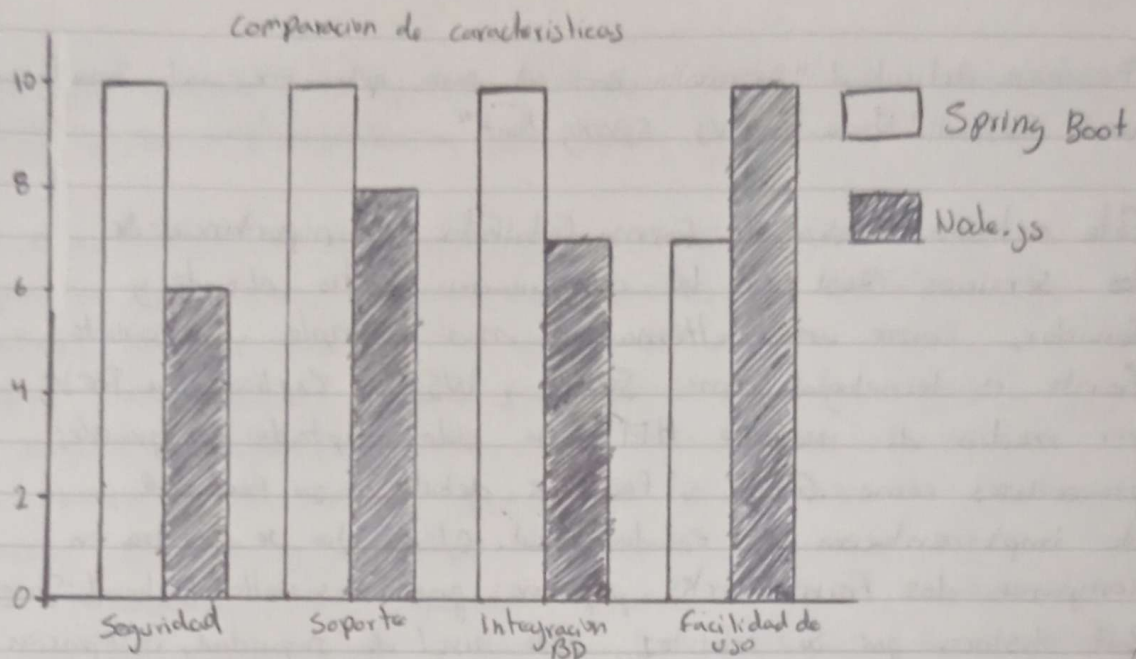


Reflexion

El Artículo muestra que los Frameworks en TS no solo agilizan el desarrollo, sino que fortalecen la formacion de los estudiantes al acercarlos a entornos laborales reales. Mas alla de la curva de aprendizaje, la verdadera ventaja esta en combinar eficiencia con una comprension solida de conceptos, preparando mejor a los futuros profesionales para afrontar proyectos reales.

Resumen Artículo 2 "Desarrollo Backend para aplicaciones web, Servicios web Restfull: Node.js vs Spring Boot"

Este artículo Analiza de forma detallada la importancia de los Servicios Rest en la comunicación entre cliente y Servidor, como una alternativa mas simple y eficiente frente a tecnologías como SOAP y WSDL, Explica que REST, por medio de metodos HTTP, ha sido adoptado por grandes proveedores como Google y Facebook debido a su facilidad de implementación y escalabilidad. El estudio se centra en comparar dos Frameworks populares para desarrollo Backend: Spring Boot destaca por su robustez, alto nivel de seguridad, integración con bases de datos SQL y herramientas empresariales; mientras que Node.js sobresale por su rapidez, facilidad de uso, capacidad asincrona y compatibilidad con bases de datos NOSQL. El Artículo describe entornos de desarrollo, herramientas utilizadas y muestra ejemplos practicos de implementación de Servicios REST en ambos frameworks. Finalmente, mediante tablas Comparativas, concluye que las dos tecnologías son utiles y eficaces, siendo Spring mas orientado a entornos empresariales por su madurez y seguridad, y Node.js mas flexible y rapido, ideal para aplicaciones que requieren Escalabilidad y tiempos de respuesta cortos. en conclusión, la elección depende del contexto del proyecto, las necesidades de integración, el tipo de base de datos y la experiencia del equipo de desarrollo

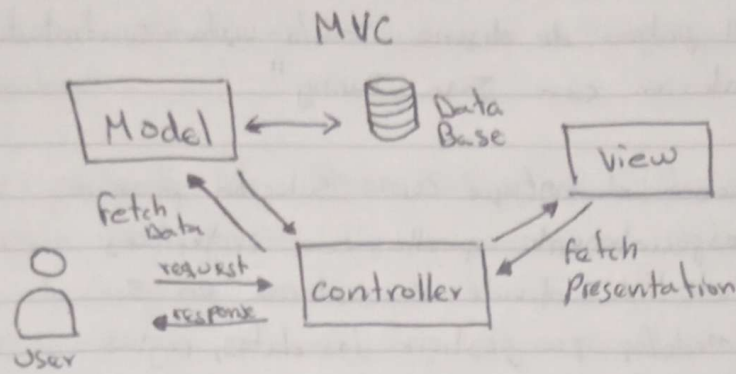


Reflexión

El artículo invita a comprender que no existe un único framework superior, sino que la elección entre Spring Boot y Node.js debe responder al contexto del proyecto. Spring aporta solidez y seguridad para entornos empresariales, mientras que Node.js brinda flexibilidad y rapidez para aplicaciones escalables y en tiempo real. La reflexión principal es que el éxito no radica en la herramienta en sí, sino la correcta alineación entre la tecnología seleccionada, los objetivos del proyecto y las competencias del equipo.

Resumen Artículo 3 "El patrón de diseño Modelo-vista-Controlador (MVC) y su implementación con Java Swing"

En este Artículo se Analiza el enfoque como solución para organizar aplicaciones, especialmente aquellas con interfaces graficas. Explica que el MVC divide el sistema en 3 Capas principales: el modelo, que gestiona los datos, reglas de negocio y estado; la vista, que muestra la información al usuario; y el controller que recibe las acciones del usuario, procesa eventos y actualiza el modelo o la vista según sea necesario. Esta separación busca reducir el acoplamiento, mejorar la escalabilidad y facilitar la reutilización de componentes. El texto describe la relación entre las capas: el controlador interpreta entradas y actúa como mediador; el modelo notifica cambios a la vista, garantizando sincronización; y la vista se mantiene simple, enfocada solo en mostrar información. El documento incluye ejemplos prácticos en Java Swing, como la gestión de una clase persona con un `PersonasTableModel` para ilustrar cómo los cambios en los datos se reflejan en tiempo real. También se presentan diagramas que aclaran las interacciones y se destacan las ventajas de MVC, como mantenibilidad y testabilidad y sus retos, como el aumento de clases y la necesidad de coordinación entre capas. El Artículo concluye que el MVC es una arquitectura robusta para proyectos medianos y grandes, recomendado su uso en entornos donde la claridad y la modularidad sean prioridades, y subrayando que, aunque exige mayor planeación, sus beneficios a largo plazo son significativos.

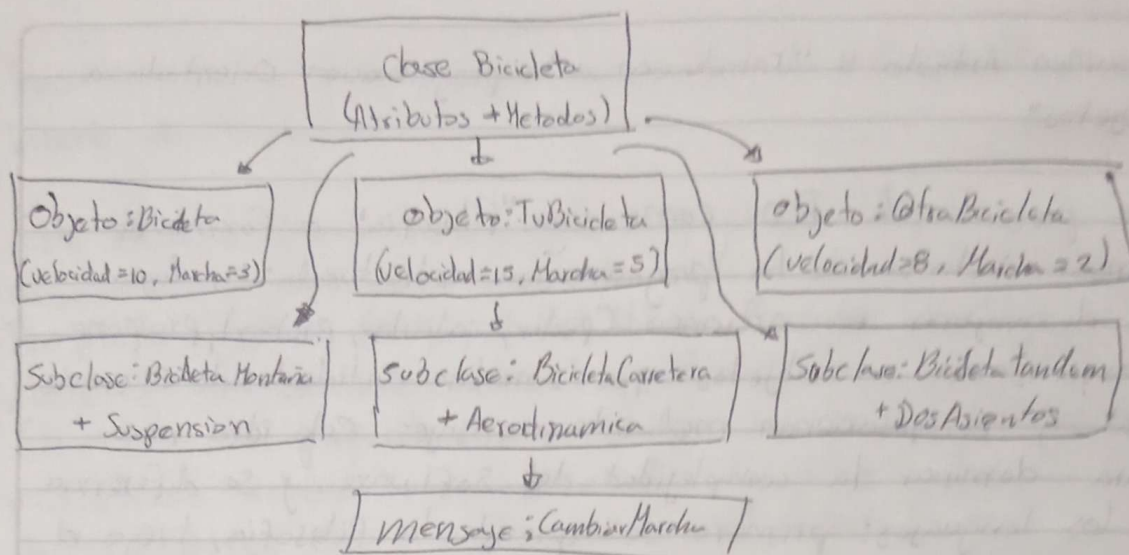


Reflexion

Este Artículo deja en claro que MVC es más que una "técnica" es una disciplina que ordena el desarrollo y garantiza software sostenible. Aunque demande mayor esfuerzo inicial y genere mas componentes, sus beneficios en claridad, escalabilidad y mantenimiento justifican la inversión, sobre todo en proyectos de mediano y gran alcance.

Resumen Artículo 4 "Introducción a la programación Orientada a Objetos"

Se presenta la POO como una "filosofía" o forma de pensar distinta de la programación estructurada: en lugar de descomponer en acciones (pedir, calcular, probar), propone descomponer en objetos que simulan entidades de escenario real y se relacionan mediante mensajes. Esta idea nace para dominar la complejidad del Software y se diferencia de los lenguajes: primero se aprende la filosofía, luego el lenguaje; de hecho, se destaca que Java es "lenguaje de objetos". Un objeto se define como datos y funciones relacionados; es la representación de un concepto del mundo real. Se enfatiza el ejemplo clásico de Bicicleta: Atributos como, velocidad, cadencia y marcha; métodos como frenar, cambiar cadencia o cambiar marcha. Este empaquetamiento de datos y operaciones ilustra la encapsulación, con distintos niveles de acceso (ej. en Java), que favorece modularidad y protección de la información. Los objetos interactúan enviándose mensajes que especifican receptor, método y parámetro (p.ejempl. CambiarDeMarcha(marcha)); esto delimita todas las posibles interacciones y permite incluso comunicación entre procesos distintos. Una clase es la plantilla que define variables y métodos comunes y de la cual se crean instancias, cada una con su propia copia de Estado. El texto usa de nuevo bicicleta para aclarar la relación clase-objeto.



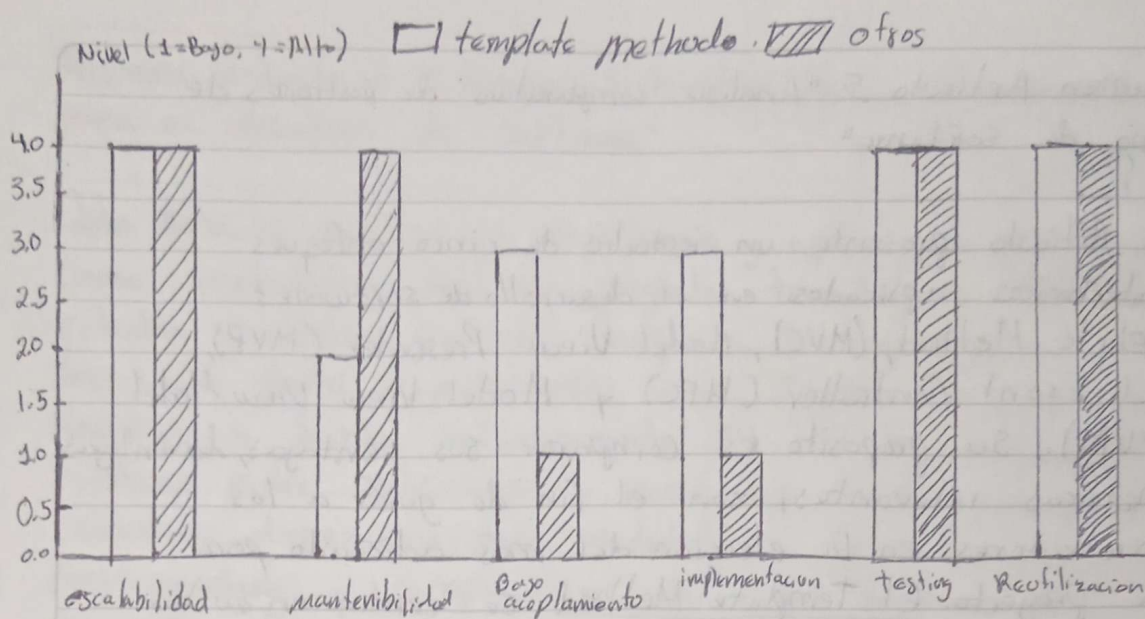
Reflexion

La POO no es solo una tecnica, sino una forma de pensar que facilita organizar el software al representar entidades reales como objetos. La metáfora de la Bicicleta ayuda a entender principios como encapsulamiento y clases, mostrando su conexión con la vida cotidiana. Mas alla de aprender un lenguaje como Java, la POO propone modularidad, comunicacion clara y extensibilidad, permitiendo construir sistemas que imitan el mundo real con precision y creatividad.

Resumen Artículo 5 "Análisis comparativo de patrones de Diseño de Software"

Este Artículo presenta un estudio de cinco enfoques arquitectónicos aplicados en el desarrollo de softwares: Template Method, (MVC), Model View Presenter (MVP), Model Front Controller (MFC) y Model View View Model (MVVM). Su propósito es comparar sus ventajas, desventajas y métricas relevantes, con el fin de guiar a los desarrolladores en la elección del más adecuado para cada proyecto. El Template Method se centra en evitar la duplicación de código mediante la definición de un algoritmo base que puede ser extendido por subclases. El MVC separa datos, lógica y presentación, lo que facilita pruebas y reutilización, aunque implica mayor complejidad. El MVP refuerza la independencia entre la vista y el modelo, facilitando pruebas unitarias, aunque su implementación es más costosa en apps pequeñas. El MFC centraliza las solicitudes de un sistema, mejorando la seguridad y organización, pero limitando la escalabilidad y finalmente El MVVM ofrece modularidad y mantenibilidad al aislar la lógica de negocio de la interfaz, aunque puede complicar la depuración.

El Análisis basado en métricas como escalabilidad, mantenibilidad, acoplamiento, implementación, testing y reutilización, demuestra que no existe un patrón superior en todos los aspectos, pues cada uno responde mejor a un contexto específico.



Reflexion

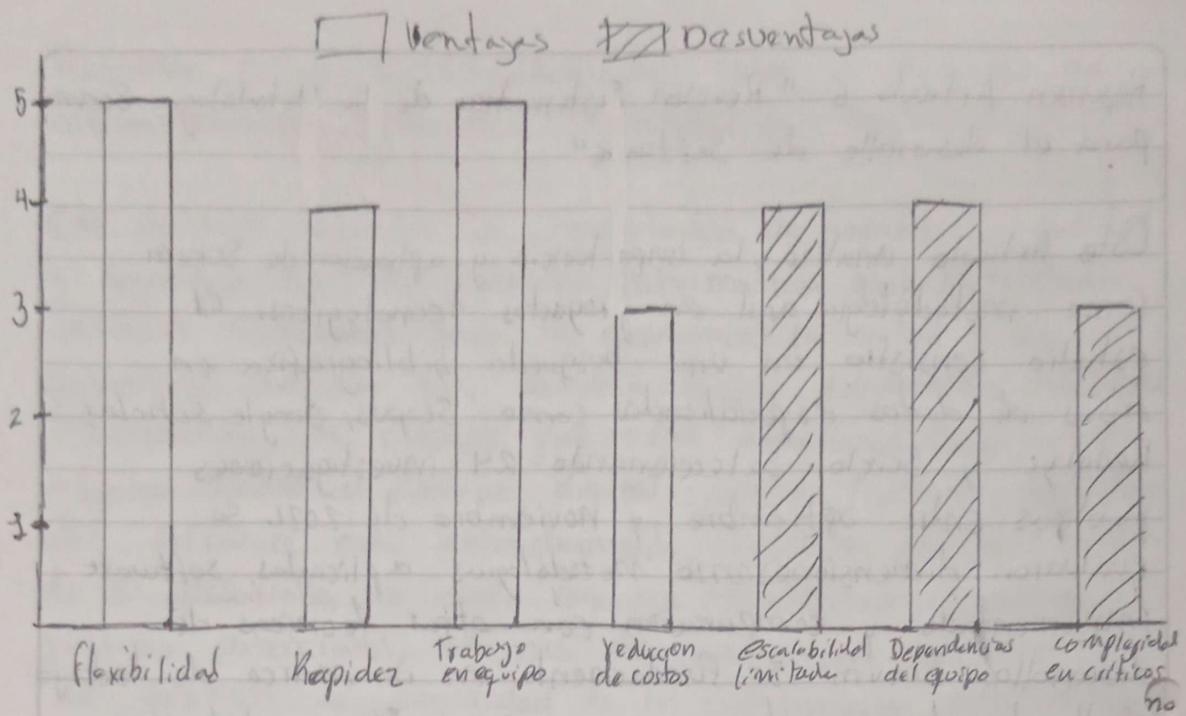
Se muestra que no existe un patrón universalmente mejor, sino que cada uno responde a necesidades distintas. Esto invita a reconocer que la verdadera habilidad no está en dominar un único enfoque, sino en saber elegir el patrón más adecuado según el contexto del proyecto y la capacidad del equipo, equilibrando simplicidad, modularidad y control.

Resumen Artículo 6 "Revisión Sistemática de la Metodología Scrum para el desarrollo de Software"

Este Artículo analiza la importancia y aplicación de Scrum como metodología ágil en proyectos tecnológicos. El estudio consistió en una búsqueda bibliográfica en bases de datos especializadas como Scopus, Google Scholar, Redalyc y Scielo, seleccionando 24 investigaciones publicadas entre septiembre y noviembre de 2021. Se evaluaron dimensiones como metodologías aplicadas, software implementado y comparación con otras técnicas de desarrollo. Scrum se fundamenta en un marco de trabajo ágil basado en transparencia, inspección y adaptación, lo que permite un desarrollo iterativo e incremental. Los resultados de la revisión muestran que Scrum es ampliamente aceptado por su flexibilidad, rapidez en la entrega de resultados y capacidad de adaptación a cambios durante el ciclo de vida del software. Además, fomenta el trabajo en equipo autoorganizado, con roles definidos como el Scrum Master y el Product Owner, que facilitan la gestión de tareas y prioridades.

Sin embargo, también se identifican limitaciones.

Scrum puede presentar dificultades en proyectos de gran escala o críticos, debido a la necesidad de precisión en los requerimientos y la dependencia de la cohesión de los equipos. Asimismo, la incorrecta conformación o conformación del grupo de trabajo puede comprometer los resultados.



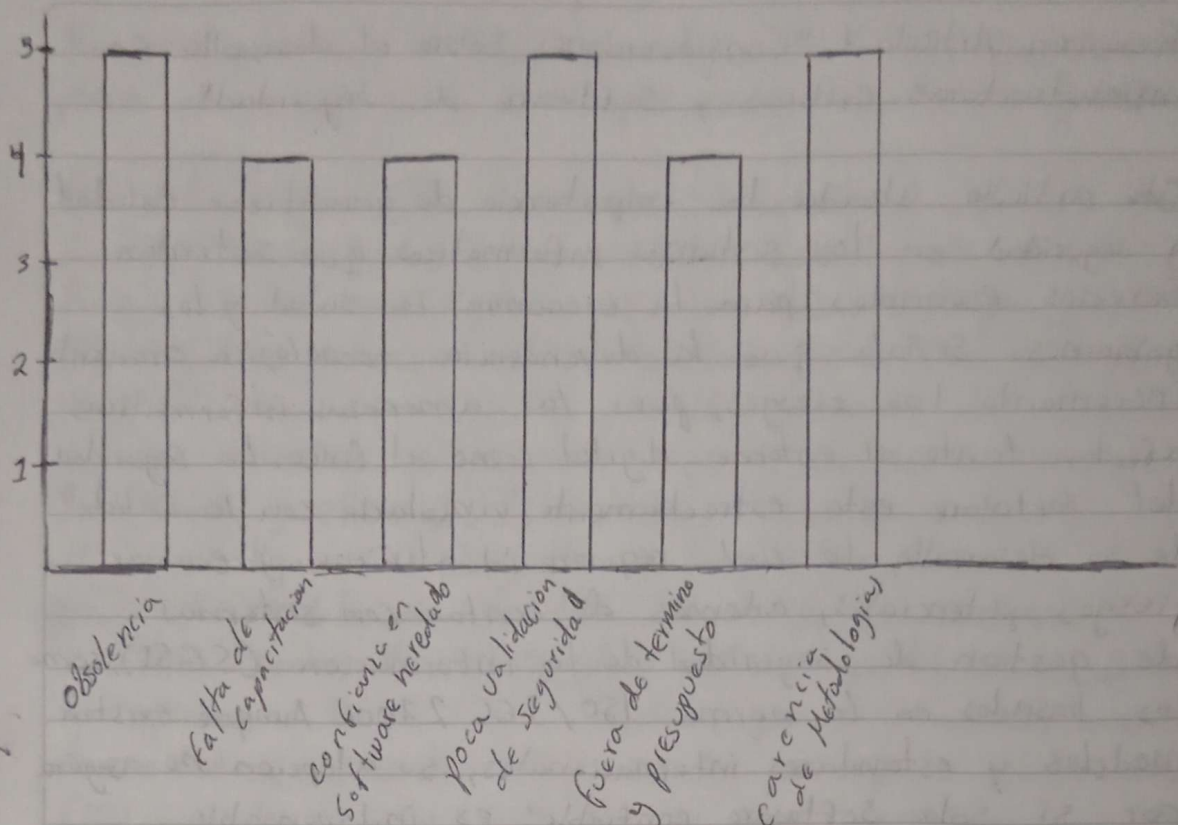
Reflexion

La Revisión Sobre Scrum invita a valorar como su fuerza radica en la colaboracion y adaptabilidad de los equipos, pero tambien recuerda que su éxito no depende solo de la metodologia, sino del compromiso humano y de elegir bien el contexto en el que aplica.

Resumen Artículo 7 "Consideraciones sobre el desarrollo en infraestructuras críticas y software de seguridad"

Este artículo aborda la importancia de garantizar calidad y seguridad en los sistemas informáticos que sustentan servicios esenciales para la economía, la salud y los gobiernos. Señala que la dependencia tecnológica creciente incrementa los riesgos, pues las amenazas informáticas afectan tanto al entorno digital como al físico. La seguridad del software está estrechamente vinculada con la calidad de su desarrollo, lo cual requiere identificar y evaluar riesgos potenciales, además de contar con sistemas de gestión de seguridad de la información (SGSI), como los basados en la norma ISO/IEC 27001. Aunque existen modelos y estándares internacionales, su adopción no asegura por sí sola software confiable; es indispensable complementarlos con metodologías, herramientas y prácticas que guíen el "cómo hacer" en el desarrollo.

El artículo expone experiencias internacionales y la situación particular de Cuba, donde se identifican problemáticas como obsolescencia de software, falta de metodologías claras, confianza excesiva en sistemas heredados, baja capacitación en seguridad y producciones fuera de término y presupuesto. Para reducir riesgos, se plantean medidas legales, físicas, lógicas, operacionales y de procesos, orientadas a garantizar confidencialidad, integridad y disponibilidad de la información.

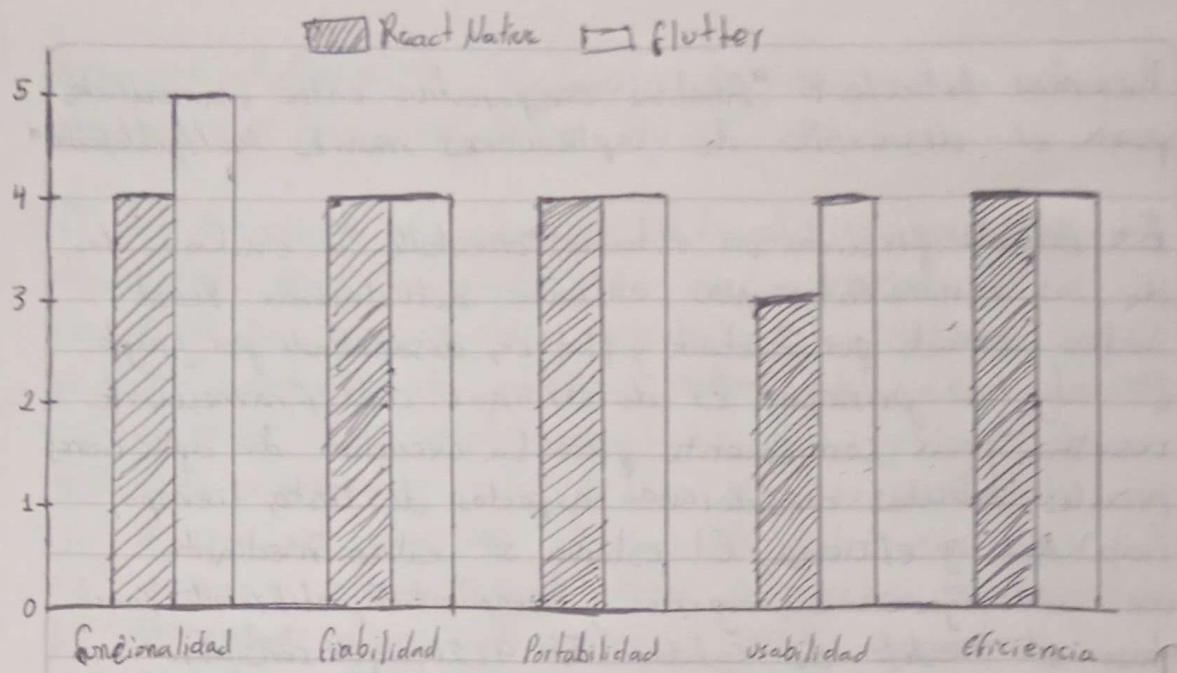


Reflexion

El Artículo nos lleva a reflexionar sobre como la seguridad en infraestructuras críticas no puede eliminarse a cumplir normas, sino que exige una cultura de prevención, capacitación y actualización constante. La confianza excesiva en sistemas heredados y la falta de metodologías claras muestran que la verdadera fortaleza está en integrar estándares con buenas prácticas y en asumir la seguridad como un compromiso continuo mas que como un requisito formal.

Resumen Artículo 8 "Análisis comparativo entre frameworks para el desarrollo de aplicaciones móviles multiplataforma"

Este Artículo presenta un estudio orientado a evaluar dos de las herramientas más utilizadas actualmente: React Native, creado por Facebook y Flutter, desarrollado por Google. El objetivo principal es determinar cual framework resulta más conveniente para la creación de aplicaciones móviles híbridas, considerando aspectos de costo, tiempo, fiabilidad y eficiencia. El estudio se realizó mediante una investigación bibliográfica y comparativa, utilizando como base el modelo de calidad ISO 9126, que considera criterios como funcionalidad, fiabilidad, portabilidad, usabilidad y eficiencia. React Native destaca por su madurez, amplia comunidad y compatibilidad con las principales plataformas (iOS y Android), ofreciendo soporte para componentes adaptativos y una integración eficiente con funcionalidades nativas. Flutter, en cambio, resalta por su enfoque innovador con widgets reutilizables, mayor facilidad de aprendizaje mediante el lenguaje Dart y la capacidad de crear aplicaciones para Android e iOS, escritorio y web con un solo código base. Ambos frameworks presentan fortalezas en documentación y soporte comunitario aunque Flutter ofrece mayor versatilidad y un enfoque más robusto en interfaces gráficas atractivas y multiplataforma. React Native mantiene ventaja para desarrolladores con experiencia previa en JS y en la librería React.

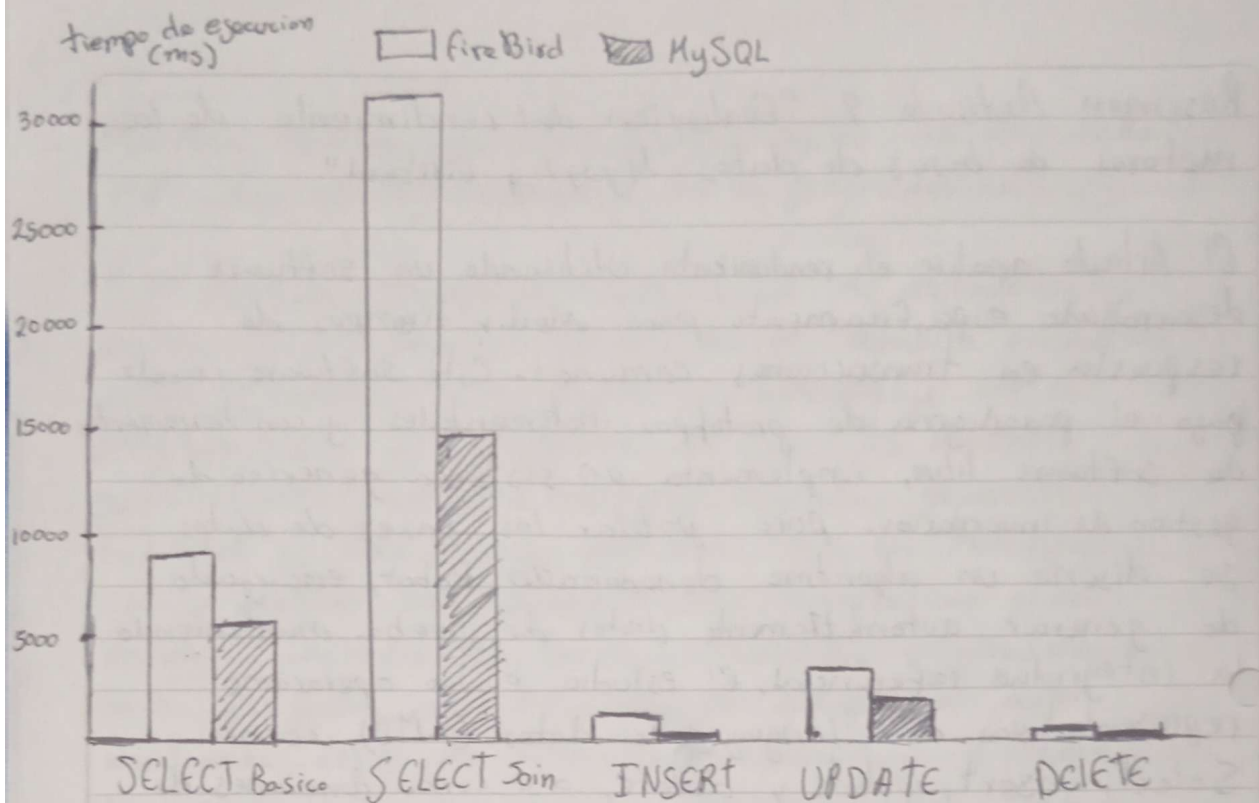


Reflexion

Se nos invita a reconocer que no existe un framework único que sea siempre la mejor opción; la elección entre Flutter y React Native depende tanto de las necesidades del proyecto como de la experiencia previa del equipo. La verdadera ventaja surge al alinear las herramientas con el contexto y los objetivos, más que en buscar una solución Universal.

Resumen Artículo 9 "Evaluación del rendimiento de los motores de bases de datos MySQL y Firebird"

El Artículo analiza el rendimiento utilizando un software desarrollado específicamente para medir tiempos de respuesta en transacciones comunes. Este software, creado bajo el paradigma de prototipos incrementales y con herramientas de software libre, implementa un sistema genérico de gestión de inventarios. Para poblar las bases de datos se diseñó un algoritmo denominado Robot, encargado de generar automáticamente datos de prueba manteniendo la integridad referencial. El estudio evaluó operaciones representativas del lenguaje de datos (LMD), como Select, insert, update y delete, así como funciones de grupo y procedimientos almacenados. Las pruebas se realizaron en entornos locales y de red, con múltiples conexiones concurrentes, lo que permitió simular escenarios cercanos a un uso real. Los resultados mostraron que MySQL superó a Firebird en la mayoría de las pruebas, destacando en velocidad de ejecución en operaciones como insert y consultas select. Firebird, aunque más lento en promedio, ofrece ventajas en cuanto a riqueza del lenguaje procedimental, manejo de disparadores y seguridad, lo que sugiere su idoneidad en aplicaciones que prioricen estas características.

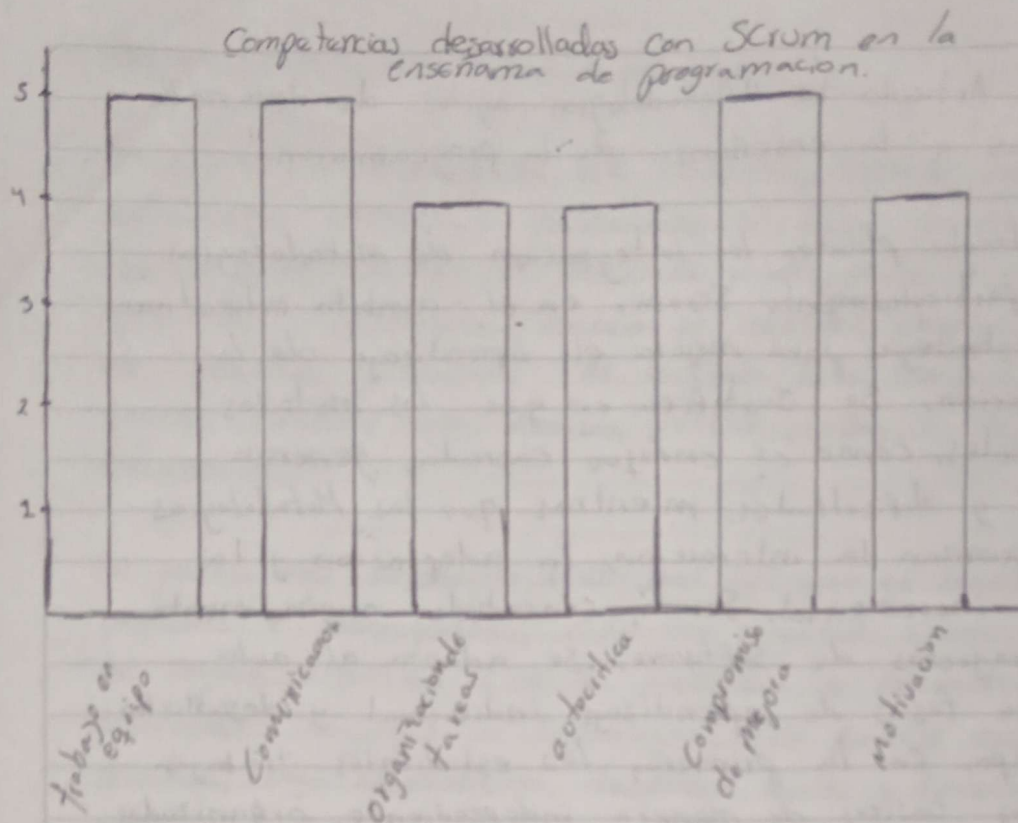


Reflexion

Este estudio muestra que elegir un motor de Base de datos no depende solo de la velocidad, sino del equilibrio entre rendimiento y las características que cada proyecto requiere. MySQL resalta en eficiencia, mientras que Firebird aporta solidez en funciones avanzadas; al final, la mejor decisión surge de alinear la herramienta con las prioridades reales del sistema.

Resumen Artículo 10 "Metodologías ágiles de desarrollo aplicadas a la enseñanza de la programación"

Este Artículo plantea la integración de metodologías ágiles, particularmente Scrum, en el ámbito educativo como estrategia para mejorar el aprendizaje de la programación. Se justifica en que los métodos tradicionales, como el enfoque cascada, generan retrasos y dificultades, mientras que las Metodologías ágiles priorizan la interacción, la adaptación y la entrega incremental. Scrum, concebido originalmente para proyectos de software, se adapta al aula mediante fases de aprendizaje individual y desarrollo en equipo. En la primera, los estudiantes trabajan contenidos básicos de manera independiente, organizados como historias de usuario y con un seguimiento a través de tableros Scrum y evaluaciones tipo test. En la segunda, se conforman equipos que desarrollan proyectos aplicando los principios de Clean Code y SOLID, bajo la dinámica de roles como Scrum Master y Product Owner adaptados a la relación entre alumno y profesorado. La propuesta fomenta competencias transversales como trabajo colaborativo, comunicación, organización de tareas, autocritica y compromiso de mejora, además de aspectos técnicos de programación. La metodología incluye sprints de dos semanas, reuniones de planificación y retrospectivas que permiten al alumnado evaluar tanto el avance del curso como el estado mínimo del grupo, garantizando así un aprendizaje integral y significativo.



Reflexión

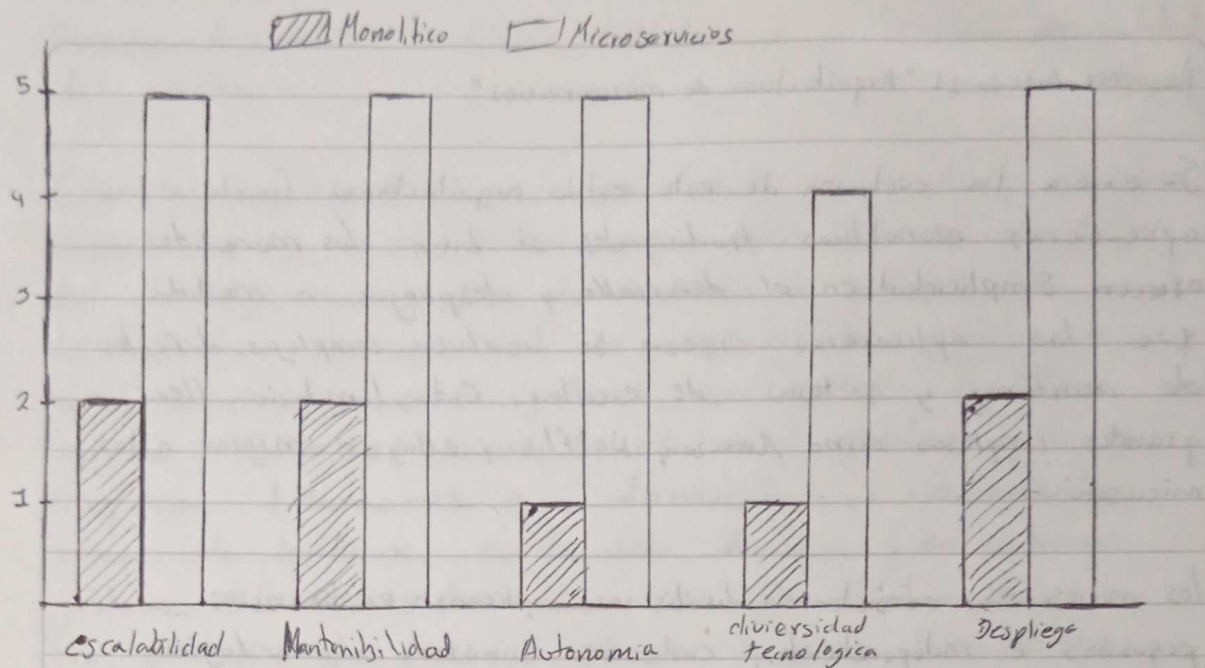
La integración de Scrum en el aula muestra que aprender programación no debe limitarse a lo técnico, sino que también puede ser una oportunidad para desarrollar trabajo en equipo, comunicación y autogestión. Al trasladar al ámbito educativo principios usados en la industria, se promueve un aprendizaje más cercano a la práctica real y, sobre todo, más significativo para los estudiantes.

Resumen Artículo 11 "Arquitectura de microservicios"

Se expone la evolución de este estilo arquitectónico frente a las aplicaciones monolíticas tradicionales. Si bien los monolitos ofrecen simplicidad en el desarrollo y despliegue, a medida que las aplicaciones crecen se vuelven complejas, difíciles de mantener y costosas de escalar. Esta limitación llevó a grandes empresas como Amazon, Netflix y eBay a migrar a los microservicios.

Los microservicios consisten en dividir una aplicación en servicios pequeños e independientes, cada uno orientado a una lógica de negocio específica y con la posibilidad de desarrollarse en diferentes lenguajes o tecnologías. Esto permite escalabilidad selectiva, bajo acoplamiento, despliegue ágiles y diversidad tecnológica, aunque introduce nuevos retos como la coordinación de múltiples servicios y la gestión de datos distribuidos.

El Artículo también describe características esenciales de este enfoque: autonomía, organización entorno a la lógica de negocio, alineación con la ley de Conway, diversidad tecnológica y la visión de productos en lugar de proyectos. Además se analiza un caso de estudio aplicado al comercio en Colombia, en el cual se propone un microservicio para la gestión de pedidos en tiendas de Barrio, con un modelo iterativo e incremental que facilite la adopción gradual de esta arquitectura.



Reflexión

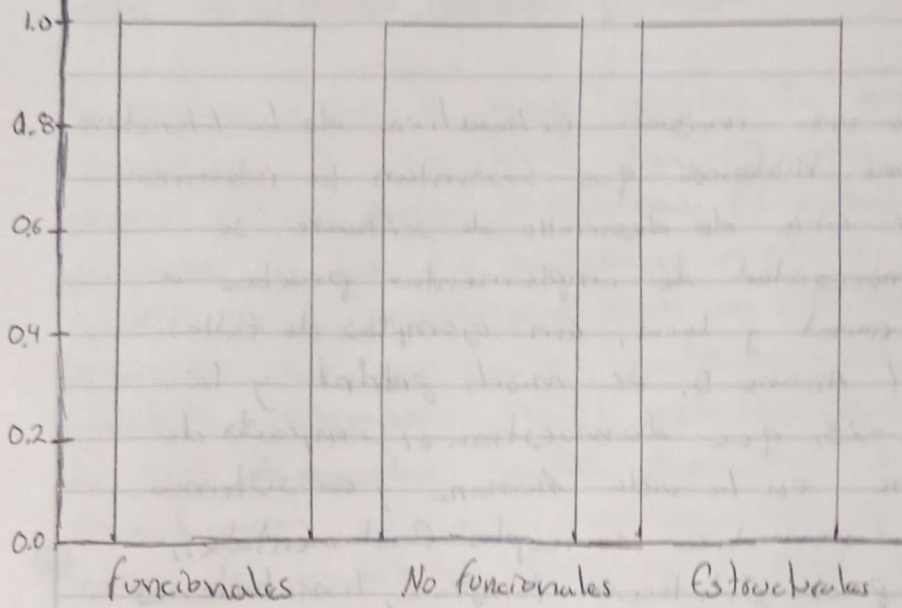
La arquitectura de microservicios invita a Reflexionar sobre como la complejidad del software moderno exige soluciones más flexibles que los monolíticos. Aunque demanda mayor gestión y conocimiento técnico, su enfoque descentralizado permite que los sistemas evolucionen al ritmo de los cambios del negocio, mostrando que la adaptabilidad es tan valiosa como la eficiencia.

Resumen Artículo 12 "Análisis del proceso de pruebas de calidad del Software"

Este Artículo ofrece una revisión exhaustiva de la literatura, experiencias y casos históricos que sustentan la relevancia del testing en el ciclo de desarrollo de software. Se contextualiza la necesidad de implementar pruebas a nivel mundial, nacional y local, con ejemplos de fallos críticos como el Ariane 5, el misil patriot y la máquina Therac-25, que demuestran el impacto de errores de software en la vida humana y en sistemas críticos. El autor sistematiza conceptos fundamentales, los principios de prueba (detección temprana, limitaciones de exhaustividad, concentración de defectos, entre otros) y presenta el modelo en V como marco de referencia para estructurar diferentes niveles de pruebas: unitarias, de integración de sistema y de aceptación. Asimismo, clasifica las pruebas en tres grandes categorías (funcionales, no funcionales y estructurales), y detalla diversas técnicas de caja negra, caja blanca, basadas en experiencia y según el responsable de ejecución. La discusión enfatiza que, pese a la abundancia de metodologías, en la práctica empresarial y académica aún se descuida el rigor en el aseguramiento de calidad, lo cual compromete la fiabilidad del software. Como perspectiva futura, se sugiere la implementación de laboratorios académicos y el fortalecimiento de la investigación aplicada en pruebas, especialmente en software móvil. El texto constituye una guía teórica y metodológica útil para empresas, universidades y desarrolladores interesados en optimizar la calidad de sus productos y reducir riesgos.

Importancia
Reactiva
(Conceptual)

Clasificación de pruebas de software Según Whittaker

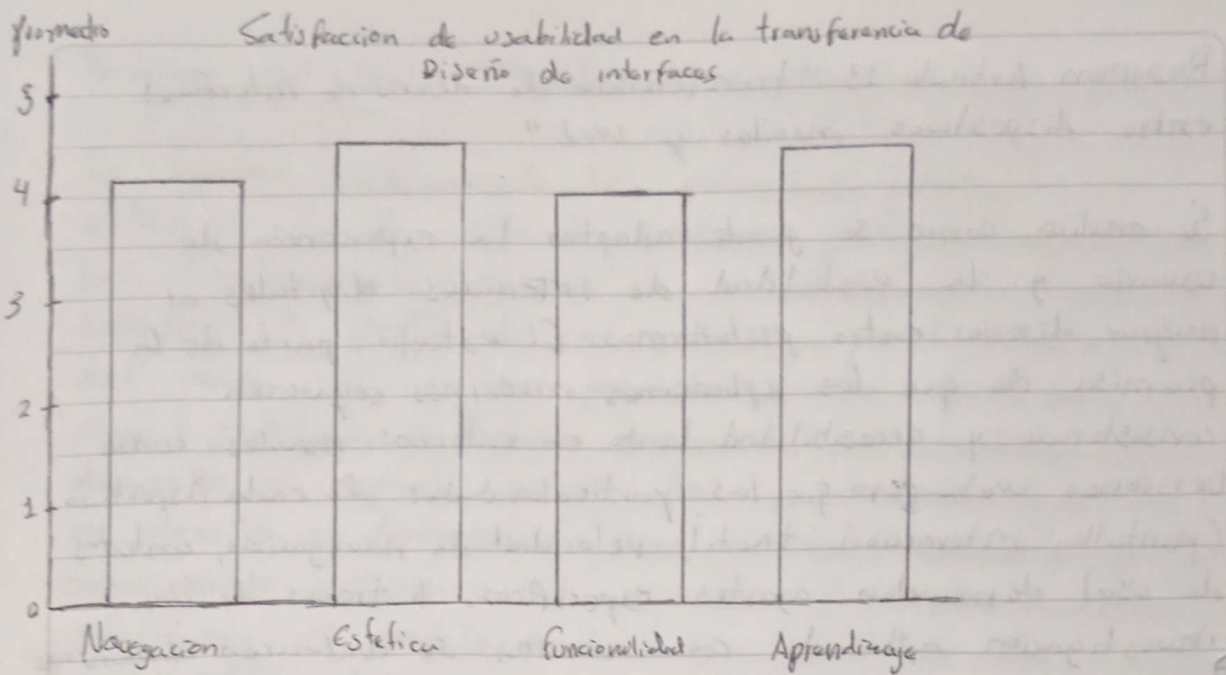


Reflexión

El artículo invita a reconocer que las pruebas no son un paso opcional ni final, sino un componente esencial para garantizar software confiable. Recordar los fallos históricos evidencia que la calidad depende de asumir el testing como una práctica continua y rigurosa. Capaz de prevenir errores costosos y de fortalecer la confianza en los sistemas que usamos a diario.

Resumen Artículo 13 "transferencia de diseño de interfaces entre dispositivos móviles y web"

Se analiza como se puede adaptar la experiencia de usuario y la usabilidad de interfaces digitales al migrar diseños entre plataformas. El estudio parte de la premisa de que las aplicaciones modernas requieren consistencia y accesibilidad tanto en entornos móviles como versiones web, pero que las particularidades de cada dispositivo (pantalla, interacción táctil, velocidad de navegación, contextos de uso) demandan ajustes específicos. A través de una investigación aplicada con usuarios, se evaluaron dimensiones clave de usabilidad como la facilidad de navegación, la estética visual, la funcionalidad percibida y la curva de aprendizaje en la adaptación de diseños. Los resultados muestran que la estética y la navegación obtuvieron las valoraciones más altas, confirmando la importancia de una coherencia visual y una estructura clara de menús, sin embargo, también se identificaron retos en la transferencia de funcionalidades avanzadas y en el aprendizaje de patrones de interacción distintos entre web y móvil. El trabajo subraya la necesidad de diseñar pensando en la portabilidad y en los contextos diferenciados de uso, proponiendo como buenas prácticas la simplificación de flujos, el mantenimiento de elementos visuales consistentes y la adaptación progresiva de funciones.



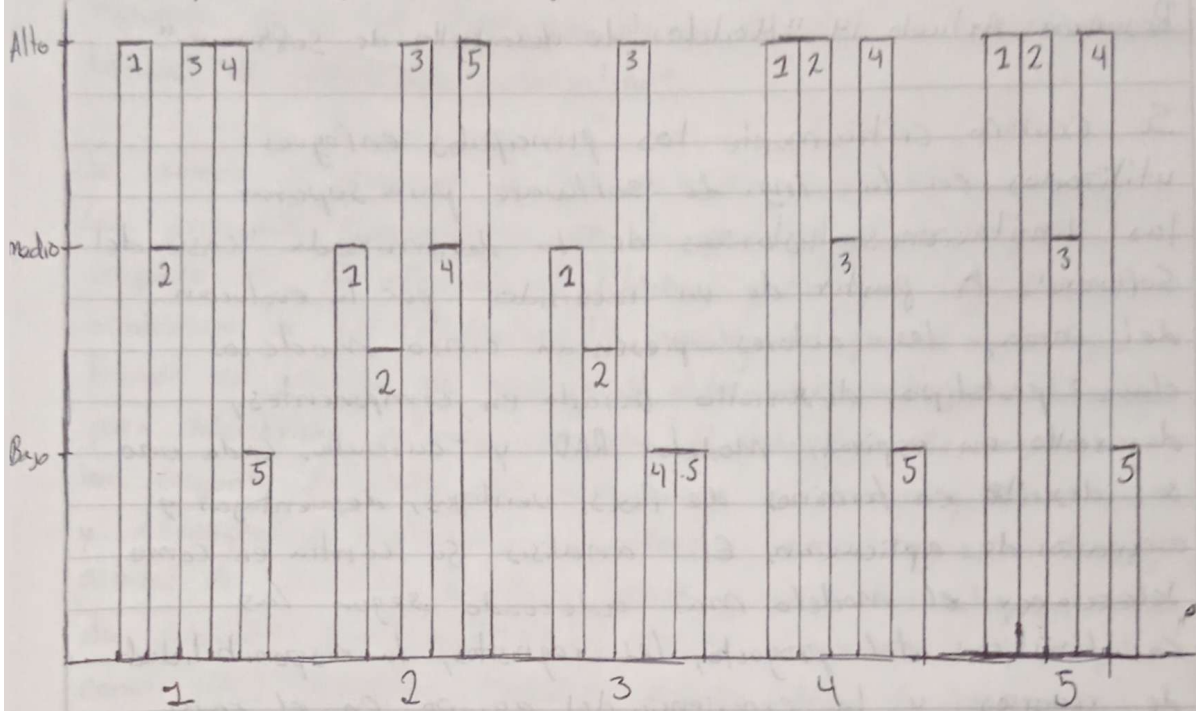
Reflexión

El Artículo muestra que diseñar interfaces no consiste solo en trasladar elementos visuales entre web y móvil, sino en comprender los contextos de uso y las formas en que las personas interactúan con cada dispositivo. La verdadera consistencia surge cuando la estética se combina con una experiencia fluida y adaptada, reduciendo la fricción y ~~pot~~ potenciando la usabilidad en ambos entornos.

Resumen Artículo 14 "Modelos de desarrollo de software"

Se examina críticamente los principales enfoques utilizados en la ing. de software para superar las limitaciones históricas de la denominada "crisis del Software". A partir de un recorrido por la evolución del área, los autores presentan cinco modelos clave: prototipo, desarrollo basado en componentes, desarrollo en espiral, Modelo RAD y cascada. Cada uno se describe en términos de fases, ventajas, desventajas y escenarios de aplicación. El análisis se centra en cómo seleccionar el modelo más adecuado según las características del proyecto, los requisitos, la disponibilidad de recursos y la experiencia del equipo. En el caso del modelo de prototipos, se destaca su utilidad para proyectos pequeños o medianos y su capacidad para clarificar requisitos, aunque con riesgos de alargar tiempos. El desarrollo en componentes se perfila como viable para entornos con alta reutilización pero condicionado a los costos y dependencias externas. El modelo en espiral, con su énfasis en la gestión del riesgo, resulta especialmente recomendado para proyectos grandes y complejos. Por su parte, RAD ofrece ciclos cortos con alta participación del usuario, aunque depende de un fuerte compromiso del equipo y recursos especializados.

1=prototipo 2=componentes 3=espiral 4= RAD 5= Cascada



1=Requisitos no predefinidos

2=Software fiable

3= Gestion de Riesgos

4= correcciones sobre la marcha

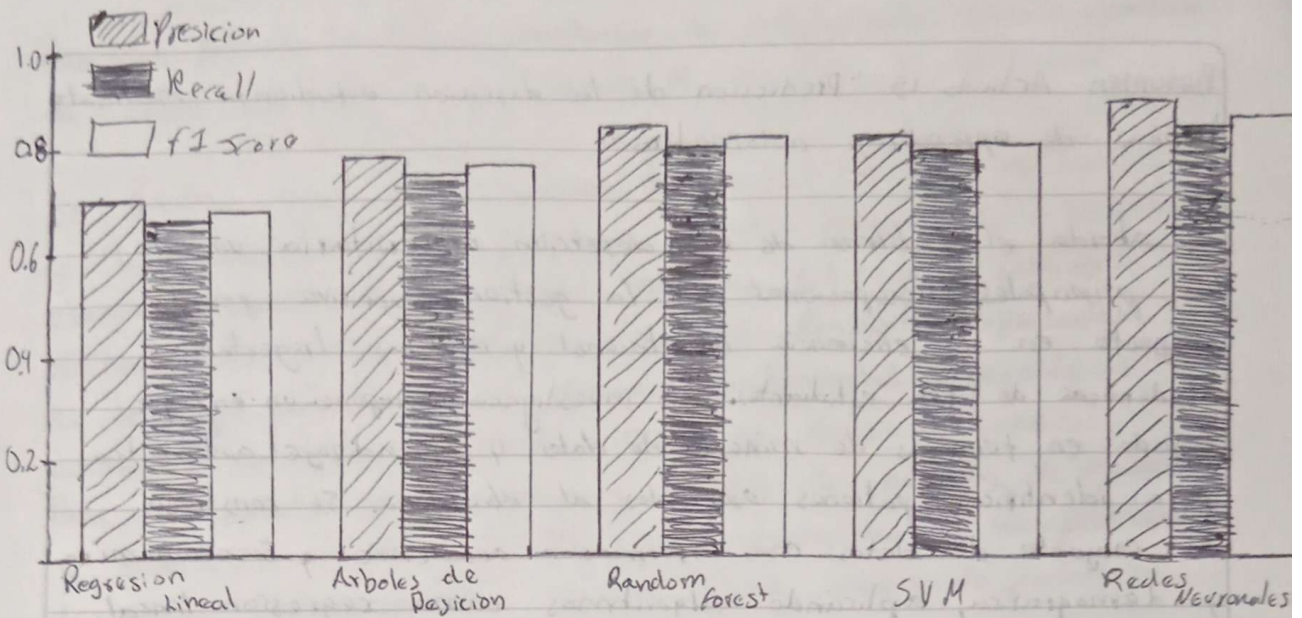
5= vision cliente / proyecto

Reflexion

El artículo invita a comprender que los modelos de desarrollo no son recetas únicas, sino herramientas que deben elegirse según el contexto. Reconocer sus ventajas y limitaciones ayuda a los equipos a tomar decisiones más conscientes, alineando el proceso con las necesidades reales del proyecto y evitando aplicar metodologías de manera rígida o automática.

Resumen Artículo 15 "Predicción de la deserción estudiantil mediante técnicas de aprendizaje automático"

Se aborda el problema de la deserción universitaria, una de las principales preocupaciones en la gestión educativa por su impacto en la eficiencia institucional y en las trayectorias académicas de los estudiantes. La investigación propone un enfoque basado en técnicas de minería de datos y aprendizaje automático para identificar patrones asociados al abandono. Se construye un conjunto de datos con información académica, Socioeconómica y demográfica, aplicando algoritmos como regresión lineal, árboles de decisión, Random forest y redes neuronales, en métricas de precisión, recall y F1-Score, frente a métodos más simples como la regresión lineal. El estudio subraya la importancia de considerar múltiples variables - rendimiento académico previo, características Socioeconómicas y nivel de compromiso Estudiantil - para mejorar la capacidad predictiva de los modelos. Asimismo, destaca que la implementación de sistemas de alerta temprana basados en estas técnicas puede contribuir a la toma de decisiones oportunas, orientadas a diseñar estrategias de intervención que reduzcan la deserción.

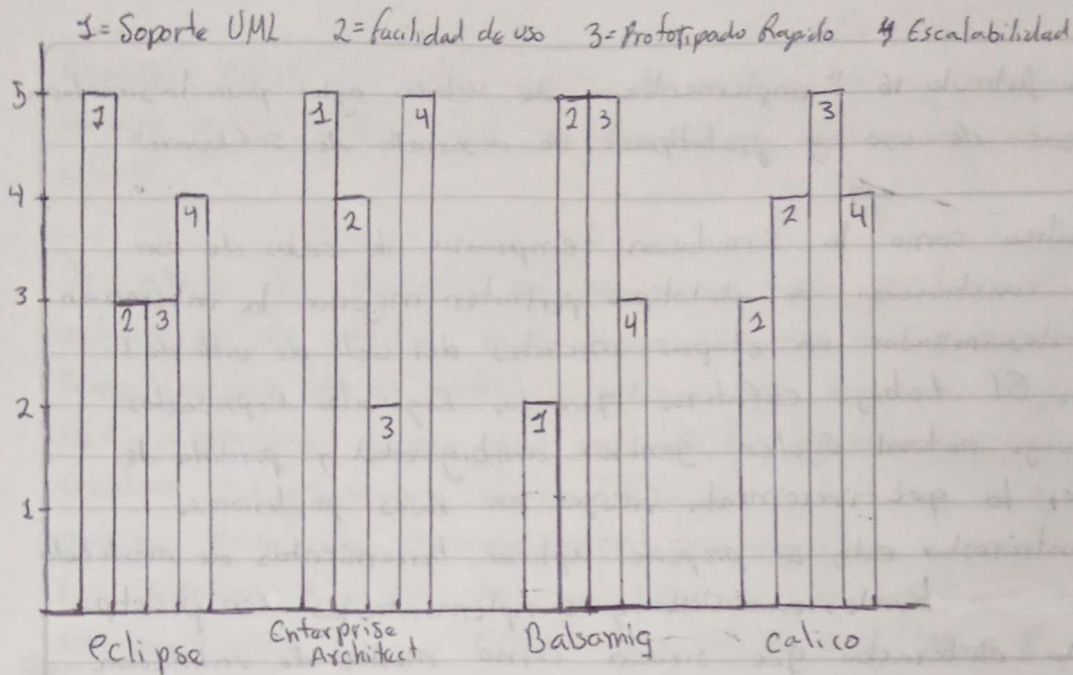


Reflexion

El estudio Resalta que comprender la desercion estudiantil exige ir mas alla de modelos simples, pues se trata de un fenomeno complejo con multiples factores interrelacionados. El uso de tecnicas avanzadas de aprendizaje automatico abre la posibilidad de anticipar riesgos y apoyar a las instituciones en la creacion de Estrategias preventivas mas humanas y efectivas, enfocadas en acompañar al estudiante antes de que abandone sus estudios.

Resumen Artículo 16 "implementación de Metodos ágiles para la simulación de casos de uso y prototipado de desarrollo de software"

Se Analiza como la Simulación temprana de casos de uso y la construcción de prototipos permiten mejorar la interacción cliente-desarrollador en etapas iniciales del ciclo de vida del Software. El trabajo enfatiza que los Requisitos expresados en lenguaje natural suelen generar ambigüedad y pérdida de contexto, lo que incrementa riesgos en fases posteriores. Para contrarrestar esto, se propone aplicar herramientas de modelado y simulación basadas en UML y complementarlas con prototipos de rápida elaboración que sirvan como medio de validación. Se comparan distintas soluciones tecnológicas como eclipse, Enterprise Architect, balsamiq y calico, evaluadas en términos de Soporte UML, facilidad de uso, escalabilidad y capacidad de prototipado rápido. El enfoque busca que el cliente participe activamente desde el inicio, validando iterativamente los modelos y reduciendo los riesgos de fallos o malentendidos en fases avanzadas. Además, se plantea que la simulación facilite la comprensión de la dinámica del sistema y la experimentación con escenarios alternativos sin necesidad de recurrir al sistema real, lo cual aumenta la eficiencia y la calidad del producto final.

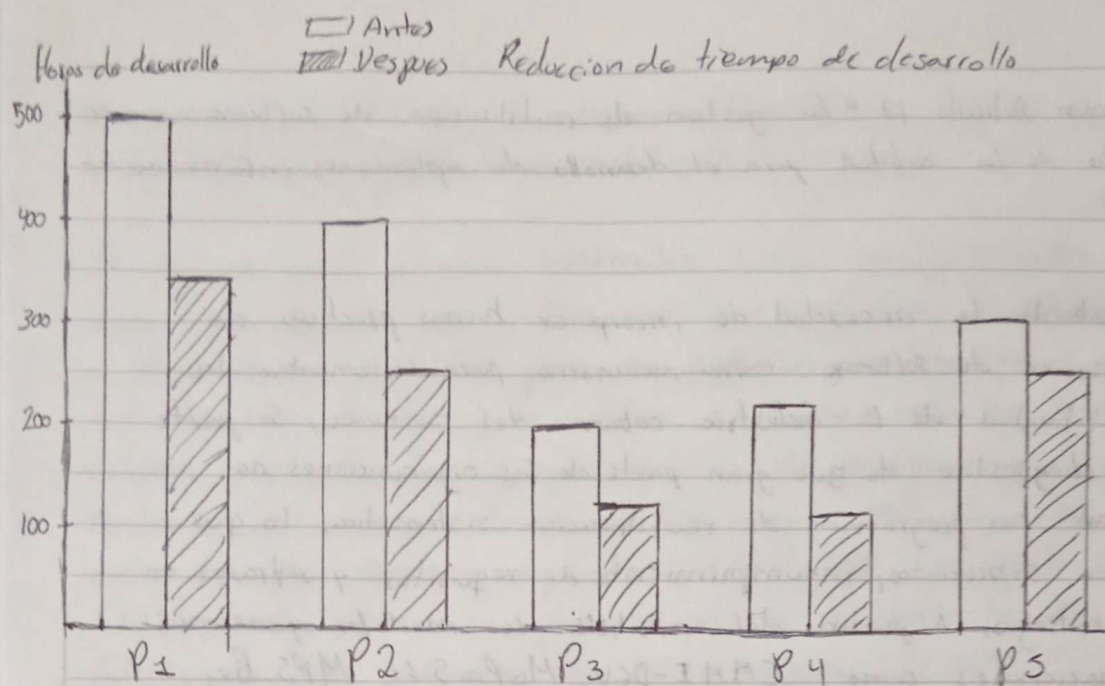


Reflexion

El artículo evidencia que la clave para mejorar la calidad del software está en involucrar al cliente desde el inicio mediante simulaciones y prototipos, reduciendo ambigüedades y riesgos futuros. Esta práctica no solo facilita la validación temprana de requisitos, sino que también fortalece la comunicación entre usuarios y desarrolladores, convirtiéndose en un puente esencial entre la visión del negocio y la implementación técnica.

Resumen Artículo 17 "La gestión de reutilización de software en el modelo de la calidad para el desarrollo de aplicaciones informáticas en Cuba"

Se aborda la necesidad de incorporar buenas prácticas de reutilización de software como mecanismo para incrementar la competitividad de la industria cubana del software. Se parte del diagnóstico de que gran parte de las organizaciones no cuentan con programas de reutilización sistemática, lo que genera sobrecostos, incumplimiento de requisitos y retrasos en las entregas. A partir del análisis de modelos y normas internacionales como CMHI-DEV, MoProSoft, MPS.Br, COMPETISOFT y las normas ISO/IEC/IEEE 12207 y 15288, los autores identifican un conjunto de prácticas clave adaptadas al modelo cubano de calidad (MCDAI). La propuesta incluye dos procesos base: Gestión de procesos de la organización (GPO) y Desarrollo de la Solución Técnica (DS), organizados en niveles de madurez (Básico, intermedio y Avanzado), lo que facilita su adopción progresiva. Para su validación, se aplicaron encuestas, grupos focales y pilotos en empresas cubanas, complementados con un estudio de caso en cinco proyectos de desarrollo web. Los resultados muestran mejoras significativas: reducción del tiempo de desarrollo y aumento en el cumplimiento de requisitos críticos. Además, el índice de satisfacción grupal alcanzó 0.85 sobre 1, reflejando alta aceptación entre especialistas.



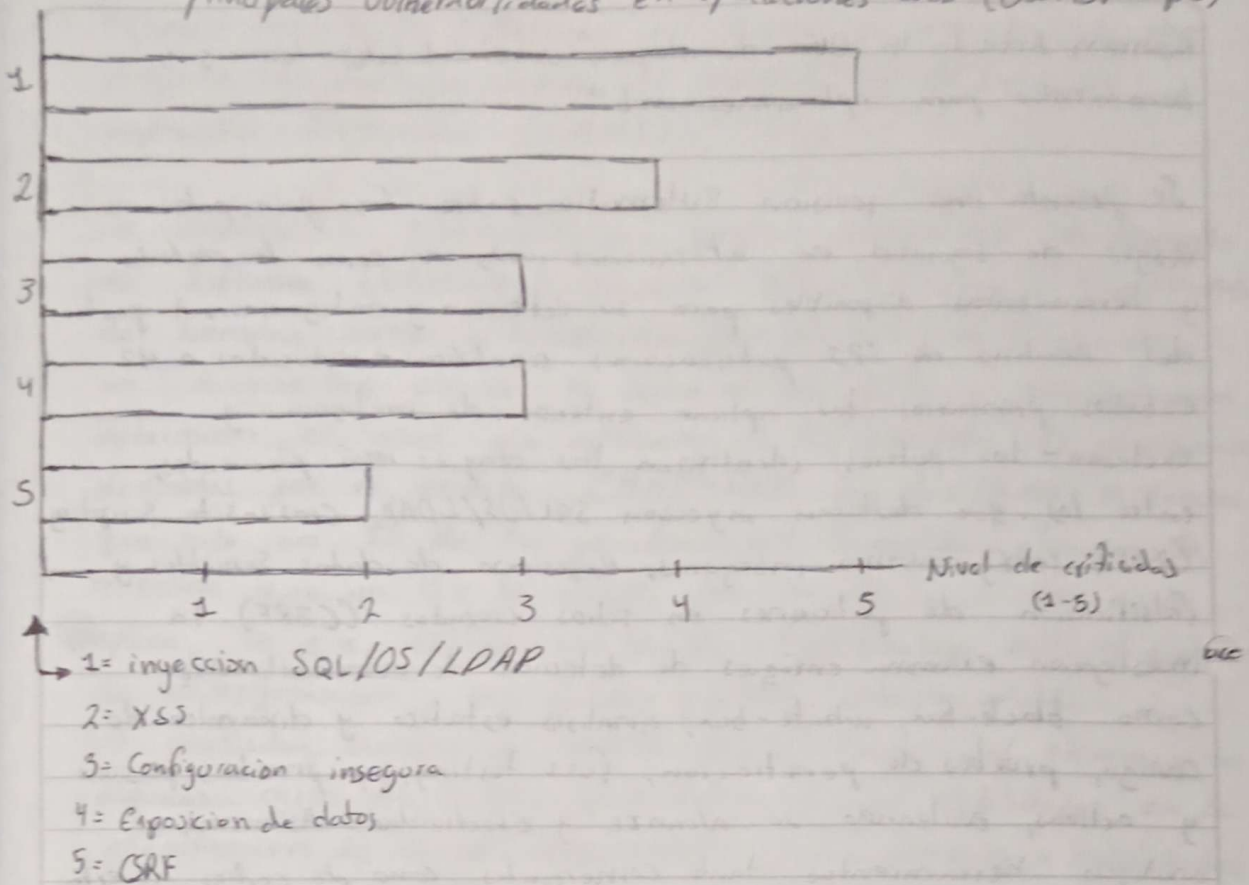
Reflexión

El estudio demuestra que la reutilización de software no es solo una estrategia técnica, sino una práctica que impacta directamente en la eficiencia y competitividad de las organizaciones. Al adoptar un enfoque progresivo y contextualizado, como el propuesto en el MCDAI, se logra aprovechar mejor los recursos, reducir tiempos y elevar la calidad, evidenciando que la sostenibilidad del desarrollo depende tanto de innovar como de reutilizar inteligentemente lo ya construido.

Resumen Artículo 18 "Guía de ataques, vulnerabilidades, técnicas y herramientas para aplicaciones web"

Se presenta una revisión sistemática sobre los principales riesgos de seguridad en aplicaciones web, así como los métodos y herramientas disponibles para su detección y mitigación. A partir del análisis de 175 publicaciones científicas - reducidas a 42 estudios primarios tras aplicar criterios de inclusión y exclusión - los autores identifican los ataques más frecuentes, entre los que destacan inyección SQL/OS/LDAP, cross-site scripting (XSS), configuraciones inseguras, exposición de datos sensibles y falsificación de peticiones en sitios cruzados (CSRF). La investigación examina enfoques de detección de vulnerabilidades como black-box, white-box, análisis estático y dinámico de código, pruebas de penetración, fuzz testing, y pruebas pasivas y activas, evaluando su alcance y efectividad. Asimismo, se analizan herramientas tanto comerciales como de código abierto, incluyendo Qualys Guard, Nessus, Retina, Web App 360, Acunetix, Burp Suite y McAfee Vulnerability Manager, entre otras. El aporte general central del trabajo es la construcción de una matriz de trazabilidad que vincula ataques, vulnerabilidades, técnicas y herramientas, proporcionando una guía práctica para seleccionar los mecanismos más adecuados de acuerdo con el tipo de amenaza.

principales vulnerabilidades en aplicaciones web (OWASP top5)

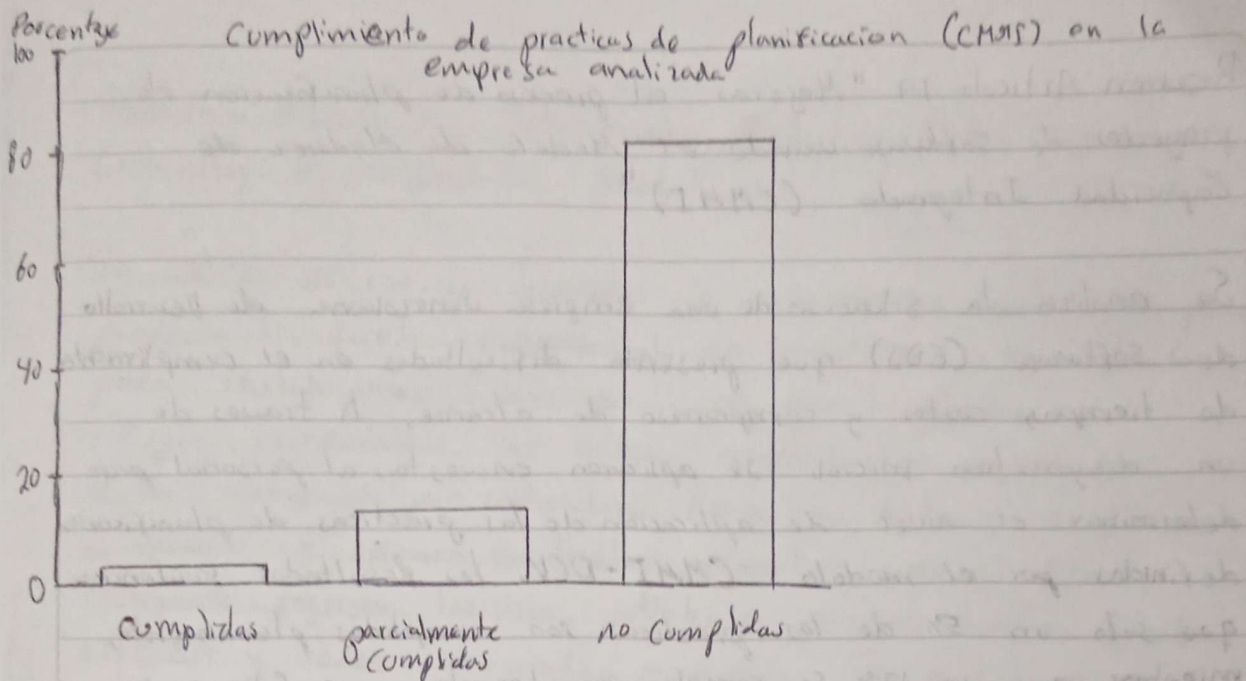


Reflexion

El Artículo pone en evidencia que la seguridad en aplicaciones web depende menos de la cantidad de herramientas disponibles y más del compromiso real de las organizaciones en aplicarlas de forma constante y adecuada. la matriz de trazabilidad se convierte en un recurso valioso porque conecta amenazas con soluciones específicas, recordándonos que la prevención efectiva surge de integrar buenas prácticas en todo el ciclo de desarrollo y no solo en fases puntuales.

Resumen Artículo 19 "Mejoras al proceso de planificación de proyectos de software usando el Modelo de Madurez de Capacidad Integrado (CMMI)"

Se analiza la situación de una empresa venezolana de desarrollo de software (EDS) que presenta dificultades en el cumplimiento de tiempos, costos y compromiso de alcance. A través de un diagnóstico inicial, se aplicaron encuestas al personal para determinar el nivel de aplicación de las prácticas de planificación definidas por el modelo CMMI-DEV. Los resultados evidencian que solo un 5% de las prácticas son cumplidas plenamente, mientras que un 12% se cumple parcialmente y un 83% no se aplica, lo que refleja una brecha significativa entre el quehacer de la organización y las buenas prácticas internacionales. En particular, aunque se realiza la estimación del alcance y en algunos casos de costos, se detectan graves debilidades en la planificación de recursos, infraestructura, conocimiento y habilidades, así como en la obtención de compromisos sobre el plan. La consecuencia ha sido el incumplimiento de plazos mediante la reducción de funcionalidades o la extensión de tiempos de entrega, afectando la satisfacción del cliente. Frente a este diagnóstico, se propone un plan de mejora estructurado en tres fases: 1. Sensibilización y capacitación del personal en prácticas de planificación, 2. implementación progresiva de las prácticas específicas de CMMI en la gestión de proyectos y 3. evaluación y retroalimentación continua para alcanzar niveles de madurez superiores.

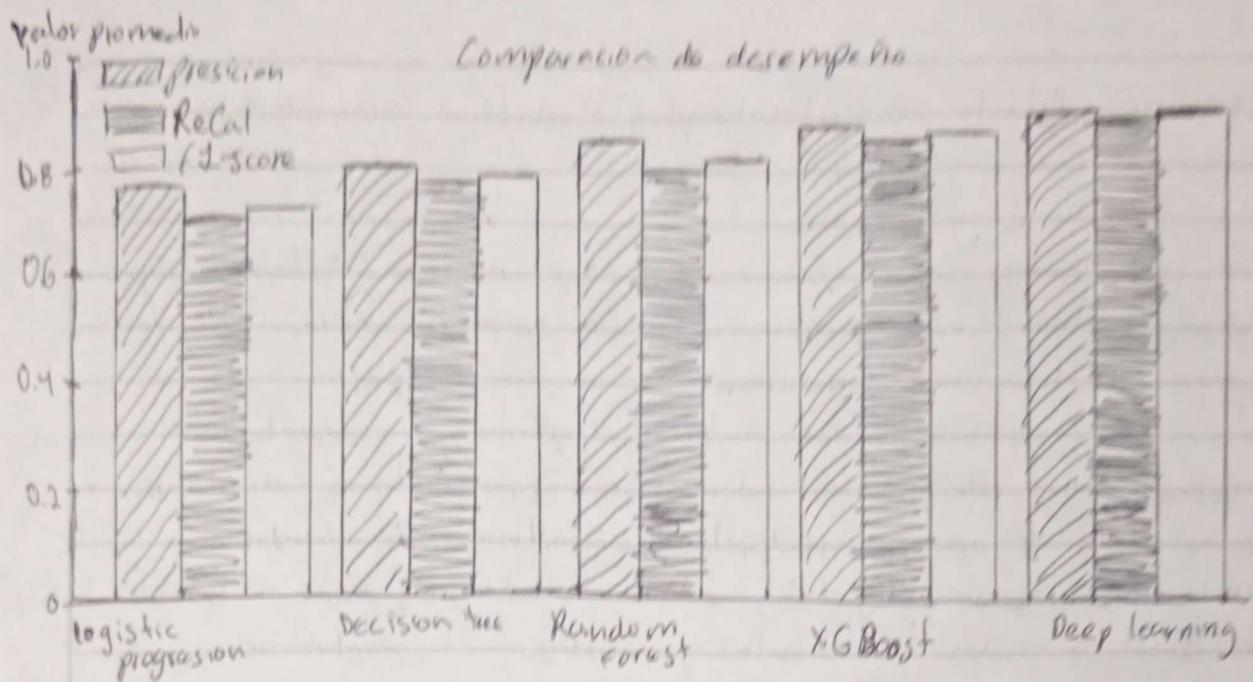


Reflexion

El caso muestra como la falta de disciplina en la planificacion puede comprometer la calidad y la satisfaccion de cliente más allá de lo tecnico. Adoptar modelos como CMPI no solo implica aplicar practicas estandarizadas, sino tambien un cambio cultural en la organizacion: Capacitar al personal, asumir la planificacion como un proceso estrategico y comprometerse con la mejora continua. De este modo la empresa no solo gana eficiencia, sino tambien confianza y sostenibilidad en el tiempo.

Resumen Artículo 20: "Bankruptcy Prediction Using Machine Learning: A Comparative Study"

Se analiza el uso de técnicas de aprendizaje automático para predecir la quiebra empresarial, un problema crítico tanto para instituciones financieras como para la gestión de riesgos comparativos. Los autores recolectaron un conjunto de datos compuesto por indicadores financieros históricos de diversas compañías y aplicaron múltiples algoritmos de clasificación, incluyendo regresión logística, árboles de decisión, Random Forest, XGBoost y redes neuronales profundas. Se evaluaron métricas de desempeño con precisión, recall y F1-score, con el fin de determinar qué modelo ofrece una predicción más fiable. Los resultados muestran que los modelos más avanzados, en particular redes neuronales y XGBoost, alcanzan los valores más altos en las métricas, superando significativamente a la regresión logística y a los árboles de decisión. Asimismo, el estudio destaca la importancia de la correcta selección de variables financieras, ya que indicadores como rentabilidad, liquidez y apalancamiento se identificaron como los predictores más influyentes. Como aporte adicional, se discuten los desafíos prácticos de aplicar estos modelos, entre ellos la necesidad de datos de calidad, el manejo de desbalance en las clases (ya que las quiebras son eventos poco frecuentes) y la interpretabilidad de los modelos complejos frente a métodos más simples.



Reflexión

El estudio resulta que predecir la quiebra empresarial va más allá de aplicar modelos simples; exige técnicas capaces de capturar la complejidad y no linealidad de las finanzas. Aunque algoritmos como XGBoost y las redes neuronales ofrecen gran precisión, también plantean retos de interpretabilidad y calidad de datos. Esto invita a reflexionar sobre la necesidad de equilibrar poder predictivo y transparencia, de modo que las instituciones puedan no solo anticipar riesgos, sino también comprender y justificar las decisiones derivadas de estos modelos.